

PALW v0.6.1：大規模言語モデル計算による ピア・ツー・ピア有用作業証明

Testnet-23 Safety Profile：正準 LLM 計算から一回使用の MatureTicket を生成する

Misaka Watanabe

info@misakachain.com

Draft v0.6.1 – 2026-08-06

要旨

ピア・ツー・ピアネットワークが信頼された調整者を置かず高コストな LLM 計算を報酬対象へ変換するには、計算が抽選前に固定され、仕事量に比例した一回限りの trial が発生し、独立 node が同じ evidence を検証できなければならない。本稿は、正準 LLM execution を一回使用の MatureTicket へ変換する PALW v0.6.1 を定義する。

Worker は同一 Compute Set・shape の複数 execution を homogeneous BatchCommit へまとめる。各 execution は projection を hiding する result salt と、lottery 専用 salt commitment を用途分離して持つ。BatchCommit 後に GHOSTDAG selected chain 上の ALGO-3 block から抽選 window を導出し、当選者だけが WinAnnounce を提出する。WinAnnounce は lottery salt、単一の winning trial、選択された job bytes、Merkle path、target reference を公開し、同一 BatchCommit に対する最初の valid announce だけが branch-local state へ受理される。検証 randomness と replica roster は winning index から独立であるため、複数当選 trial の選択が監査・replica grinding を生まない。

仕事の真正性は三層で検証する。層 0 の Non-Interactive Sampled Audit (NISA) は、将来乱数で選ばれた execution interval を full node が再実行する validity 条件である。層 1 では二つの大額 bond を持つ replica B/C を経済合理的主体として扱い、primary の projection 公開前の blind replay、全 consensus field を束縛する署名、署名後の追加 future audit、客観 timeout と fault evidence により不正を検知する。層 2 では PendingTicket の challenge window 中に誰でも permissionless dispute を開始でき、bisection を単一 Compute VM micro-step まで進める。Accepted evidence は投票や裁量を介さず branch-local state で必ず freeze、suspend、slash settlement へ進み、全 dispute が解決し state が安定した後にはのみ MatureTicket が生成される。

Testnet-23 Safety Profile では ALGO-3 が唯一の ordering と liveness を担い、PALW ticket は fork-choice weight、transaction ordering 権、実価値 coinbase を持たない。MatureTicket は zero-value conformance record として deterministic に自動 finalization される。非ゼロ ordering weight を持つ ALGO-4 profile は本稿の規範範囲外であり、別の安全性証明、GHOSTDAG 再校正、独立監査、hard fork を要する。この fail-closed な分離により、未校正の useful-work 経済が chain ordering を購入する経路を持たない。

1 規範範囲と配備 profile

本稿は PALW Core と Testnet-23 Safety Profile を規範的に定義する。PALW Core の出力は一回使用の MatureTicket である。MatureTicket は、抽選前に固定された execution commitment、将来 randomness、NISA、bonded replay、permissionless dispute を通過したことを表す。

1.1 Testnet-23 Safety Profile

Testnet-23 の consensus parameter は次を強制する。

項目	規範値
Ordering	ALGO-3 selected chain のみ
PALW fork-choice weight	0
PALW transaction ordering 権	無し
PALW coinbase / transferable reward	0
MatureTicket	zero-value conformance record
Replica slash	shadow slash ledger。実 UTXO burn は無効
DNS / validator authority	consensus input として使用禁止
Compute Set	genesis allowlist。完全 artifact と ComputeVM package を node が保持

したがって、Testnet-23 で PALW object に欠陥があっても、ALGO-3 の fork choice、通常 transaction state、coinbase

supply を変更できない。BatchCommit、WinAnnounce、replica message、opening、certificate、dispute はすべて ALGO-3 control transaction またはその consensus adjunct として運ばれ、maturity/finalization は ALGO-3 state transition として決定的に実行される。Public testnet の目的は、window reconstruction、deterministic runtime、witness、replica assignment、dispute、reorg rollback、IBD recovery、resource budget を測定することである。

v0.6.1 は旧 PALW wire、BLAKE2b/u128 target、OpeningAnnounce 型と互換ではない。Testnet-23 は fresh genesis、専用 network identifier、専用 subnetwork byte、v0.6.1 RulesetID を使用し、既存 network の object を in-place で再解釈してはならない。

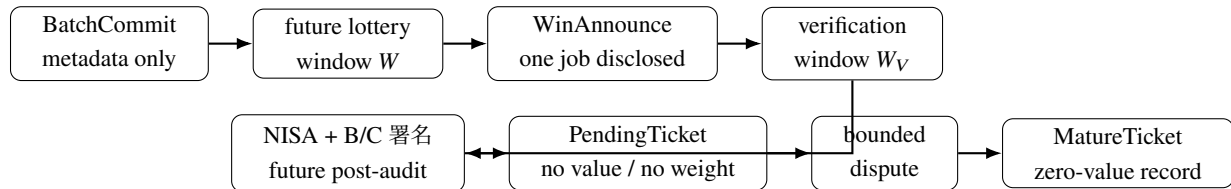


図 1: v0.6.1 の唯一の規範 flow。B/C 署名後にも将来乱数監査を行い、FinalCertificate と MatureTicket より前に ordering、transaction effect、transferable reward は存在しない。

1.2 Value Profile と Ordering Profile

Transferable reward を持つ Value Profile は、15 節の gate と別 monetary-policy specification が成立するまで無効である。Value Profile は B/C を利他的主体ではなく、大額 bond を失うことを避ける経済合理的主体として扱う。したがって activation は、検知確率の保守下限、最大 bribe・ticket gain、実損となる slash、credential suspension cost を用いた replica rationality inequality を満たす場合に限る。非ゼロ fork-choice weight、ALGO-4 transaction execution、block-template binding を持つ Ordering Profile は本稿に含めない。これらを有効化する実装は v0.6.1 準拠ではなく、別 protocol version として再監査しなければならない。

2 設計原則

- P1. 一 execution、一 commitment。** Authority sequence と leaf index は canonical input に入り、同じ execution を別 leaf、別 sequence、別 BatchCommit へ載せ替えると別計算になる。
- P2. 仕事先行の抽選。** Projection commitment は lottery window より前に chain へ受理される。Win 後に初めて計算して commitment を開くには hash preimage を破る必要がある。
- P3. lottery salt と result salt の用途分離。** WinAnnounce で公開する lottery salt は projection を検査可能にしない。Result salt は replica commitment 後まで非公開である。
- P4. winner-only data availability。** 非当選 job bytes は chain に載せない。選択 job bytes は WinAnnounce carrier に必須添付され、欠落した announce は invalid である。
- P5. 単一の pre-finality flow。** Certificate は PendingTicket を生成し、dispute 終結後にのみ MatureTicket になる。MatureTicket より前に reward、ordering、transaction effect は存在しない。
- P6. branch-local state。** Sequence、first-wins、capacity、bond、dispute、ticket、nullifier、budget は candidate block の parent state に対して評価し、reorg で巻き戻す。
- P7. 全 resource の集約上限。** Per-object cap だけでなく、selected-chain epoch ごとの bytes、verifier cost、control transaction、active dispute、roster utilization を consensus が制限する。
- P8. fail-closed activation。** ComputeVM package、wire vector、runtime conformance、resource benchmark のいずれかが欠ける profile は activation value が zero のままである。

3 暗号 primitive と正準 encoding

Protocol hash H は domain-separated SHA3-512 であり、各 preimage は protocol tag、wire version、network identifier、RulesetID、固定順の length-delimited field を含む。Target comparison は digest の先頭 256 bit を big-endian unsigned integer として解釈する。Consensus object に float、host-endian integer、map の非正準順序、unknown field を含めてはならない。

Authority と replica signature は ML-DSA-87 を用い、object class ごとに異なる context string を固定する。Canonical

Merkle tree は leaf count、tree arity、odd-leaf rule、path direction bit を wire contract で固定し、root だけで異なる長さの sequence を表現できない。Hash、signature、Merkle、fixed-point arithmetic、rejection sampling の positive/negative vector は RulesetID に含まれ、独立二実装が一致しない ruleset は activation できない。

4 Homogeneous BatchCommit

4.1 Batch descriptor と canonical input

BatchCommit は同一 Compute Set s 、同一 shape σ の N execution を束ねる。Heterogeneous batch は v0.6.1 で無効である。Batch descriptor は

$$D = (\text{network}, \text{ruleset}, h_A, \text{seq}, A_0, s, \sigma, N, r_{\text{reward}}) \quad (1)$$

を含む。 h_A は ML-DSA-87 authority public key commitment、 A_0 は selected-chain ALGO-3 anchor、 r_{reward} は将来の Value Profile が用いる reward script commitment であり Testnet-23 では zero script に固定する。

Authority sequence は worker 自己申告の自由値ではなく、parent state の NextSequence[h_A] と完全一致しなければならない。BatchCommit 受理は sequence を一だけ増加させ、同一 pair を持つ二つ目の transaction は無効である。Anchor A_0 は BatchCommit parent から Δ_{anchor} ALGO-3 selected-chain block 以内の ancestor でなければならない。Batch seed は

$$R_{\text{batch}} = H(\text{palw}/\text{batch-seed}/v1 \parallel D) \quad (2)$$

であり、leaf $j \in \{0, \dots, N-1\}$ の canonical execution freshness は

$$R_{\text{work},j} = H(\text{palw}/\text{work}/v3 \parallel R_{\text{batch}} \parallel \text{u32be}(j)) \quad (3)$$

である。Runtime は $R_{\text{work},j}$ を protocol-defined input position へ挿入するため、同じ prompt を複数 leaf へ複製しても同一 execution にはならない。

4.2 Execution commitment と salt 分離

Execution j の projection を

$$P_j = (r_{\text{exec},j}, r_{\text{out},j}, n_{\text{tok},j}, e_j, a_{\text{op},j}) \quad (4)$$

とする。 $a_{\text{op},j}$ は全 semantic operation を順序付きで束縛する operation accumulator の最終値である。

A は互いに独立な 256-bit value $\rho_{\text{res},j}$ と batch-level lottery salt ρ_{lot} を生成する。Result commitment と lottery commitment は

$$C_{\text{exec},j} = H(\text{palw}/\text{exec-commit}/v1 \parallel j \parallel R_{\text{work},j} \parallel r_{\text{in},j} \parallel r_{\text{ctx},j} \parallel P_j \parallel \rho_{\text{res},j}), \quad (5)$$

$$C_{\text{lot}} = H(\text{palw}/\text{lot-commit}/v1 \parallel \rho_{\text{lot}}) \quad (6)$$

である。Job leaf は

$$L_j = H(\text{palw}/\text{job-leaf}/v1 \parallel j \parallel r_{\text{in},j} \parallel r_{\text{ctx},j} \parallel C_{\text{exec},j}) \quad (7)$$

とし、 $r_{\text{jobs}} = \text{MerkleRoot}(L_0, \dots, L_{N-1})$ を得る。Batch identifier は

$$\text{BC} = H(\text{palw}/\text{batch-commit}/v1 \parallel D \parallel r_{\text{jobs}} \parallel C_{\text{lot}}) \quad (8)$$

である。

ρ_{lot} を WinAnnounce で公開しても $C_{\text{exec},j}$ は独立な $\rho_{\text{res},j}$ により hiding のままである。逆に、 ρ_{lot} は C_{lot} へ独立に束縛されるため、window 後の salt 差し替えはできない。

4.3 署名、fee、one-shot rule

BatchCommit signing preimage は byte-level wire contract で固定し、少なくとも

$$(\text{version}, \text{network}, \text{ruleset}, D, \text{BC}, r_{\text{jobs}}, C_{\text{lot}}, f_{\text{attempt}}, h_{\text{expiry}}) \quad (9)$$

を覆う。RC 自体を署名対象から漏らす旧形式は無効である。

BatchCommit は non-refundable attempt fee のみを消費する。Fee は

$$f_{\text{attempt}} = f_0 + f_{\text{byte}}|\text{BatchCommit}| + f_q q_{\text{batch}} \quad (10)$$

であり、fake leaf を用いて fee を薄めることはできない。Work escrow と replica-fee escrow は WinAnnounce 時に初めて lock する。これにより、非当選 attempt が大量の native token を長期間 lock することはない。

Branch-local state は $\text{batchState}[BC] \in \{\text{Committed}, \text{Announced}, \text{Expired}\}$ と $\text{lotCommitUsed}[C_{\text{lot}}]$ を持つ。 C_{lot} は同一 branch 上で一度だけ登録でき、lottery salt は commit から announce まで秘密かつ fresh でなければならない。BatchCommit は一度だけ WinAnnounce へ遷移でき、同一 branch 上で二件目の announce は無効である。Lottery window 終端から Δ_{announce} service opportunity 内に announce が無ければ state は Expired となり、active index と一時 metadata は prune できる。

各 shape の trial quantum 数 $q_{\text{job}}(s, \sigma)$ は登録 semantic program と opcode cost vector から node が再導出する。Batch の総 trial 数は

$$q_{\text{batch}} = N q_{\text{job}}(s, \sigma) \quad (11)$$

であり、 $N \leq N_{\text{max}}$ 、 $q_{\text{batch}} \leq q_{\text{max}}^{\text{batch}}$ を満たす。Worker 自己申告の q は consensus input にならない。

Global T_{max} は

$$q_{\text{max}}^{\text{batch}} \frac{T_{\text{max}} + 1}{2^{256}} \leq \varepsilon_{\text{multi}} \quad (12)$$

を満たし、初期 $\varepsilon_{\text{multi}} = 2^{-10}$ とする。これにより一 batch 内の複数 win は稀であり、one-ticket cap による quantum 当たり機会の非線形性を有界化する。

4.4 未計算 leaf を混ぜる戦略

未計算 leaf は valid $C_{\text{exec}, j}$ opening を持たない。その leaf に対応する trial が当選しても A は WinAnnounce 後の opening を完成できず、work escrow を失う。未計算 leaf は計算済み leaf の trial 値を増やさず、BatchCommit は一 ticket しか生成しないため、未計算 leaf の追加は正当な block chance を増やさない。これは attempt fee と winner escrow により有界な失敗戦略であり、ordering へ影響しない。

5 Canonical execution, state tree, Compute VM

5.1 Full node が保持する artifact

v0.6.1 の full node は有効な Compute Set の model weights、tokenizer、semantic program、opcode table、shape table、state-tree schema、conformance vector をローカルに保持する。Interval witness に weight tile 本体を埋め込まない。Genesis と upgrade parameter は、active Compute Set 数 S_{max} と全 weights 合計 $B_{\text{weights}}^{\text{max}}$ を上限化する。

これにより checkpoint 間隔を witness tile size に合わせて sub-matmul 粒度へ細分化する必要がある。Weight availability は activation package の取得・hash 検証で解決し、block validity は外部 online fetch に依存しない。

5.2 Semantic operation と operation accumulator

Compute VM package は各 opcode の typed input、integer range、overflow rule、read/write set、canonical encoding、state transition、one-step verifier、cost を完全に固定する。Package hash を ComputeVMID とし、ruleset はこれを含む。Package、positive/negative golden vector、独立二実装が存在しない ruleset は activation できない。

実行 state は少なくとも KV cache、activation、routing、output、token counter、stop reason と operation accumulator a_t を leaf とする。各 semantic operation o_t に対し

$$a_{t+1} = H(\text{palw/op-acc/v1} \parallel a_t \parallel \text{enc}(o_t) \parallel \text{enc}(\text{reads}_t) \parallel \text{enc}(\text{result}_t) \parallel \text{enc}(\text{writes}_t)) \quad (13)$$

を更新する。Checkpoint 前に上書きされる scratch、破棄される MoE path、padding operation も accumulator へ入る。したがって状態境界に現れない operation を省略しても checkpoint root は一致しない。

5.3 Fixed-op execution と checkpoint

Shape は正確に $\text{ops}(s, \sigma)$ cost unit を実行し、

$$q_{\text{job}} = \frac{\text{ops}(s, \sigma)}{\kappa_q}, \quad q_{\text{cp}} = \frac{\text{ops}(s, \sigma)}{\kappa_{\text{cp}}} \quad (14)$$

を満たす。 κ_q と κ_{cp} は両方とも $\text{ops}(s, \sigma)$ を割り切り、partial interval は存在しない。

Decode が自然停止した後も、runtime は latched output を保持しつつ deterministic padding program を実行する。Admission は各 shape の padding cost 比率を $u_{\text{pad}}^{\text{max}}$ 以下に制限し、shortcut analysis は padding 領域を含む全 operation budget を対象にする。

Checkpoint root を $D_0, \dots, D_{q_{\text{cp}}}$ とし、

$$r_{\text{exec}} = \text{MerkleRoot}(D_0, \dots, D_{q_{\text{cp}}}) \quad (15)$$

とする。Interval witness は pre/post checkpoint、touched state leaves、Merkle paths、opcode slice を含み、weights は node の local artifact を参照する。

6 Selected-chain randomness と target

6.1 Batch 固有の future window

Randomness source は GHOSTDAG selected chain 上の ALGO-3 block のみである。BatchCommit carrier hash を B_c とし、window offset を

$$\delta(\text{BC}) = \text{U256}(\text{H}(\text{palw/window-offset/v1} \parallel \text{BC} \parallel B_c)_{0..31}) \bmod w_{\text{spread}} \quad (16)$$

とする。v0.6.1 は w_{spread} を 2 の冪に限定し、modulo bias を持たない。 B_c は transaction inclusion 前に authority が知れないため、authority は BatchID の hash grinding だけで endpoint を選べない。

Selected chain 上で $k + \delta(\text{BC})$ 個の ALGO-3 block を待ち、続く w block を ordered MMR へまとめる。

$$W = (B_c, A_{k+\delta}, \dots, A_{k+\delta+w-1}, r_{\text{MMR}}) \quad (17)$$

同じ carrier に入った BatchCommit も BC ごとに endpoint が分散するため、一つの boundary block の withholding が全 in-flight batch を同時に再抽選する効果は w_{spread} により抑えられる。

6.2 WinAnnounce-rate と capacity controller

Lottery target は challenge 後の block rate ではなく、selected chain 上で受理された WinAnnounce 数を制御する。Controller anchor を (m_a, h_a, T_a) 、現在までの accepted WinAnnounce sequence を $m(h)$ 、ALGO-3 selected-chain service height を h 、目標 WinAnnounce interval を I_W とし、

$$T_{\text{rate}}(h) = \text{clamp}\left(T_a 2^{\frac{h-h_a-(m(h)-m_a)I_W}{H_{\text{half}}}}, T_{\text{min}}, T_{\text{max}}\right) \quad (18)$$

を fixed-point integer arithmetic で評価する。 $H_{\text{half}} \geq 10\Delta_{\text{announce}}$ を consensus 条件とし、target freeze から WinAnnounce 受理までの dead time に対して十分に遅い controller とする。

Lottery window 終端の reserved-slot ratio を U_{ref} とする。Capacity clamp は fixed-point で

$$T_{\text{cap}}(U) = \begin{cases} T_{\text{max}}, & U \leq U_{\text{soft}}, \\ T_{\text{max}} \frac{U_{\text{max}} - U}{U_{\text{max}} - U_{\text{soft}}}, & U_{\text{soft}} < U < U_{\text{max}}, \\ 0, & U \geq U_{\text{max}} \end{cases} \quad (19)$$

と定義する。Ticket target は

$$T_{\text{ticket}} = \min(T_{\text{rate}}(h), T_{\text{cap}}(U_{\text{ref}})) \quad (20)$$

である。 $U_{\text{ref}} \geq U_{\text{max}}$ の場合は admission-closed flag も立ち、hash が zero であっても WinAnnounce を受理しない。Replica capacity が飽和したとき target を緩める正帰還は存在しない。

Lottery window 終端は

$$\text{ref}_T = (\text{blockID}, \text{stateRoot}, \text{controllerVersion}, \text{fixedPointID}, U_{\text{ref}}) \quad (21)$$

を固定する。 ref_T と T_{ticket} は WinAnnounce、certificate、PendingTicket、MatureTicket の全署名 preimage へ入る。Live target を後から読む手順は存在しない。

6.3 Private trial と一回限りの WinAnnounce

Lottery randomness と trial key は

$$R_{\text{job}} = H(\text{palw/job-randomness/v5} \parallel \text{BC} \parallel W), \quad (22)$$

$$v_{\text{lot}} = H(\text{palw/trial-key/v3} \parallel \rho_{\text{lot}}), \quad (23)$$

$$X_i = H(\text{palw/trial/v3} \parallel R_{\text{job}} \parallel \text{BC} \parallel \text{u64be}(i) \parallel v_{\text{lot}}) \quad (24)$$

である。比較には SHA3-512 output の先頭 256 bit を big-endian U256 として用いる。A は $0 \leq i < q_{\text{batch}}$ のうち $X_i \leq T_{\text{ticket}}$ を満たす任意の一 index を announce できる。BatchCommit は一 ticket しか生成せず、verification randomness は winning index と job leaf から独立なので、複数 win の選択は replica · audit draw を変えない。

Index i は

$$j = \left\lfloor \frac{i}{q_{\text{job}}} \right\rfloor, \quad u = i \bmod q_{\text{job}} \quad (25)$$

により選択 job と quantum を一意に定める。

WinAnnounce は

$$\text{WA} = (\text{BC}, h_A, \rho_{\text{lot}}, i, j, u, \text{ref}_T, T_{\text{ticket}}, r_{\text{jobBytes}}, \pi_{\text{leaf}}, E_w, E_{\text{rep}}) \quad (26)$$

を含む。Authority signature は canonical WinAnnounce 全体、network、RulesetID、escrow output を覆う。Carrier は selected job の public input/context bytes を consensus adjunct として必ず運ぶ。Full node は C_{lot} opening、leaf path、shape からの q 、index mapping、 $X_i \leq T_{\text{ticket}}$ 、signature、escrow、epoch win cap、capacity permit を即時検証する。

同一 block 内の WinAnnounce は canonical transaction index 順に処理し、各受理 transition が capacity と budget を atomic に予約する。Capacity permit が無い transaction は valid state transition ではなく block へ含められないため、first-wins state も escrow lock も生じない。Branch-local transition

$$\text{announced}[\text{BC}] : \perp \longrightarrow H(\text{WA}) \quad (27)$$

は最初の valid transaction だけを許す。

6.4 検証 window と roster reference

Replica roster root r_{slots} は lottery window 終端の branch state から固定し、WinAnnounce carrier 時点を使わない。これにより A は announce timing で roster composition を選べない。

WinAnnounce carrier 後に k_2 block を待ち、続く w_2 ALGO-3 block を verification window W_V とする。Randomness は

$$R_{\text{rep}} = H(\text{palw/replica-randomness/v5} \parallel \text{BC} \parallel \text{carrier}(\text{WA}) \parallel W_V \parallel r_{\text{slots}}), \quad (28)$$

$$R_{\text{audit}} = H(\text{palw/audit/v5} \parallel \text{BC} \parallel \text{carrier}(\text{WA}) \parallel W_V) \quad (29)$$

である。Winning index、job bytes hash、projection、result salt はこの randomness へ入れない。

7 Data availability と consensus resource budget

非当選 job bytes は chain に載らない。WinAnnounce carrier は selected job bytes を運ぶ。Primary opening、二つの replica reveal、post-audit reveal、FinalCertificate が複数 control transaction に分かれる場合も、同一 ticket に属する canonical evidence の合計は $B_{\text{pending}}^{\text{max}}$ 以下でなければならない。FinalCertificate transition はそれらの hash と availability completion を束縛し、PendingTicket はその FinalCertificateID を参照する。外部 online fetch は block validity の前提にならない。

各 ALGO-3 block は通常 transaction budget と分離した PALW control payload 上限 B_{ctrl} を持つ。Deadline を持つ object class には deterministic weighted round-robin calendar が service slot を割り当てる。一つの scheduled class は当該 block で最大 B_{ctrl} を使用でき、未使用分は他 class が借用できるが、service clock は canonical calendar によってのみ進む。通常 fee market や他 class の spam は予約 service opportunity を消費できない。BatchCommit は deadline class ではなく、残余枠で処理する。

WinAnnounce、replica commitment、primary opening、replica reveal、post-attestation audit reveal、ReplicaEvidence reveal、challenge reveal、dispute response、ChallengeOverflowSignal の各最大 object は B_{ctrl} 以下でなければならない。より大きい evidence format は activation 不可であり、黙って複数 block に分割して別 timeout 規則を作ってはならない。Testnet-23

の calendar cycle は 16 ALGO-3 block とし、上記 class は各 cycle に少なくとも 1 slot、dispute response と ReplicaEvidence は追加 reserve slot を持つ。Exact slot map は parameter package と golden vector へ含める。

Budget epoch は class ごとの非借用 reserve $B_{\text{epoch},c}$ と $V_{\text{epoch},c}$ を持ち、その総和は global cap 以下でなければならない。他 class と BatchCommit は deadline class の epoch reserve を消費できない。各 class は eligible object を (deadline, scopelD, objectKind, objectHash) で並べる deterministic fair queue を持つ。scopelD は BatchID、PendingTicketID、gameID の該当値である。Calendar slot は queue 先頭の object へ service turn を割り当て、その object の deadline clock だけを一進める。別 ticket の transaction が slot を消費して honest object の時計だけを進めることはない。

Class-specific service turn は、calendar slot が存在し、当該 class の reserved byte/cost budget にその object の最大 envelope を受理できる残量がある場合にだけ発生する。これにより protocol 自身の aggregate cap が honest response の時計を進めながら inclusion を不可能にすることはない。

時間軸は二つだけである。Lottery window、announce expiry、challenge window、retention、stable depth は ALGO-3 selected-chain height で数える。Slashable response deadline は object ごとに割り当てられた service turn count で数える。Ruleset は calendar cycle 長 g_{cycle} と各 class の最大 slot gap g_c を固定し、 n opportunity の wall-clock 上限を ng_c selected-chain block として再導出できなければならない。Producer の local clock、DAA score、外部 epoch は用いない。

Consensus は selected-chain budget epoch E_B ごとに次を検査する。

$$\sum \text{adjunctBytes} \leq B_{\text{epoch}}, \quad (30)$$

$$\sum \text{VerifierCost} \leq V_{\text{epoch}}, \quad (31)$$

$$\# \text{BatchCommit} \leq C_{\text{batch}}, \quad (32)$$

$$\# \text{WinAnnounce} \leq C_{\text{win}}, \quad (33)$$

$$\# \text{newDispute} \leq C_{\text{dispute}}. \quad (34)$$

Per-object cap または epoch cap のいずれか一方でも、当該 object を追加した結果として超える場合、その object は invalid である。したがって許容上限同士を掛けた無制限の帯域・CPU・retention は発生しない。

Timeout clock は object-specific service turn 数で進む。Object が fair queue 上で turn を割り当てられない高さでは時計は進まず、割り済み turn で producer が valid transaction を継続的に除外する場合だけ targeted hash-lane censorship と判定する。通常 transaction congestion と PALW class starvation は同じ故障として扱わない。

8 Layer 0: NISA

8.1 Sample の一意性

NISA index は R_{audit} から rejection sampling で重複なく抽出する。続く replica B、C の index は固定順に抽出し、NISA と先行 replica の全 index を skip する。したがって

$$\tau_{\text{eff}} = \tau + 2\kappa_r + 2\kappa_p \quad (35)$$

個の random interval は構成上すべて相異なる。ここで κ_p は B/C が署名した後の post-attestation audit で各 replica に追加される区間数である。先頭・末尾 boundary interval はこれに追加される。

8.2 Opening と witness

Primary opening は selected leaf の $(r_{\text{in}}, r_{\text{ctx}}, P, \rho_{\text{res}})$ 、Merkle path、checkpoint roots、NISA witness を含む。Full node は

- (i) leaf path と C_{exec} opening、
- (ii) $R_{\text{work},j}$ と initial state、
- (iii) T_{ticket} と WinAnnounce の winning trial、
- (iv) sampled interval の κ_{cp} operation、
- (v) operation accumulator、first/last boundary、output projection

を検証する。削除済み candidate target や block 時点の live target を参照してはならない。

Interval witness は

$$\Pi_t = (D_t, D_{t+1}, L_t, \pi_t^{\text{state}}, \pi_t^{\text{exec}}, \text{opcodeSlice}_t) \quad (36)$$

であり、weights は active Compute Set package から読む。Witness size と one-interval verifier cost は shape admission 時に固定する。

9 Layer 1: Bonded replica verification

9.1 Roster, capacity, canonical assignment

Replica lease は credential、Compute Set、shape、bond、performance slot、correctness lien slot、activation height、exit state を持つ。Lottery window 終端時点で年齢 Δ_{age} を満たす active lease だけが roster root に入る。

WinAnnounce 受理時、network は二つの provisional global capacity token を予約する。Global reserved ratio と credential ごとの reserved ratio は

$$U_{\text{global}} = \frac{\text{reservedSlots}}{\text{totalActiveSlots}} \leq U_{\text{max}}, \quad U_c = \frac{\text{reservedSlots}_c}{\text{activeSlots}_c} \leq U_{\text{max}} \quad (37)$$

を満たさなければならない。WinAnnounce 受理時は U_{global} を、具体的な B/C が確定する assignment 時は U_c を検査する。超える transition は受理されない。初期 $U_{\text{max}} = 1/4$ とする。各 credential の nominal performance slot capacity $S_c = \lfloor B_{\text{total}}/B_{\text{perf}} \rfloor$ は自己申告でなく node が再導出し、同時に collateral inequality を満たさなければならない。 $U_c = \text{reservedPerfSlots}_c/S_c$ とする。各 honest credential の free fraction が少なくとも $1 - U_{\text{max}}$ に保たれるため、attacker だけが空きを温存した場合も free-slot share の増幅は有界である。

W_V 終端で R_{rep} から roster の permutation を導出し、primary と同一 credential を除外し、branch-local に free な performance slot を順に取って B/C を選ぶ。同一 block で複数 assignment が成立する場合の順序は (W_V end height, BC) の辞書順である。Provisional token はこの transition で actual B/C reservation へ atomic に変換し、選出不能なら解放して ticket path を fail closed に終了する。短い list へ縮退せず、二つの distinct credential が得られなければ certificate を作らない。

9.2 二段 bond reservation

Selection 時に各 replica は B_{perf} を予約する。これは full replay、commitment、reveal、availability を担保し、FinalCertificate 成立または客観 failure まで lock される。B/C が CB へ署名した時点で、その内側の B_{corr} は correctness sub-lien として release 不可になる。FinalCertificate 成立後は $B_{\text{perf}} - B_{\text{corr}}$ を解放し、 B_{corr} を PendingTicket の correctness lien として maturity または rejection まで保持する。

$$\sum_{\text{active assignments}} B_{\text{perf}} + \sum_{\text{pending certificates}} B_{\text{corr}} \leq B_{\text{rep}} \quad (38)$$

を authority ごとに強制する。Proven false certificate は correctness lien を slash し、credential を suspend し、同一 bond が支える他の pending certificate を freeze する。Value Profile では B_{corr} が ticket 最大価値以上で、かつ $p_{\text{det}}^{\min} S_R^{\min} > G_R^{\max}$ を満たすよう hard-fork parameter で定める。Testnet-23 は transferable UTXO を burn しないが、shadow amount、credential suspension、capacity removal、freeze transition は実際の branch-local state へ適用する。

Lease の exit request は直ちに新規 roster eligibility を停止するが、bond release は performance reservation、correctness lien、pending SlashReport がすべて zero となり、最後の署名済み certificate から $R_{\text{arch}} + \Delta_{\text{stable}}$ を経過するまで実行できない。証拠期間中に bond を逃がす経路は存在しない。

No-show は assignment、required object class、assigned service turn、deadline height、未消費 response ID を含む自己完結 evidence である。Credential ごとの rolling epoch no-show count が $N_{\text{ns}}^{\max}$ に達すると lease を suspend し、cooldown と capacity top-up を終えるまで selection set へ戻さない。

9.3 経済前提と replica rationality

Value Profile は B/C が大額 bond を失うことを避ける経済合理的主体であると仮定する。この仮定は「B/C は不正をしない」という宣言ではなく、不正を検知する複数経路と、検知後に必ず実行される slash により次を満たすという activation condition である。

$$G_R^{\max} < p_{\text{det}}^{\min} S_R^{\min}. \quad (39)$$

$G_R^{\max}$ は replica が受け取り得る最大 bribe と ticket-related gain、 $p_{\text{det}}^{\min}$ は mandatory audit と watcher exercise から得る保守的検知下限、 $S_R^{\min}$ は該当 fault で実行される slash、credential suspension、capacity loss、exit delay から coalition が回収できる bounty 等を差し引いた最小実損である。Value Profile はこの不等式を実測値と最大値で満たせない限り activation できない。Testnet-23 は real burn を行わないため、この合理性を証明せず、検知と shadow enforcement の完全性だけを試験する。

9.4 Evidence completeness と署名対象

各 replica obligation は AssignmentID、role、object kind、expected body hash、service-turn deadline を branch-local state に登録する。期限までに canonical outer signature を持つ response が存在するか、存在しなければ objective timeout が発生する。曖昧な silence 状態は存在しない。

Replica envelope は

$$\text{REnv}_R = (\text{network}, \text{ruleset}, \text{branchAnchor}, \text{AssignmentID}, \text{role}, \text{objectKind}, \text{bodyHash}, \text{deadline}, \sigma_R) \quad (40)$$

である。Outer signature 自体が invalid な byte 列は第三者が捏造できるため、それだけで slash してはならず、response として無効にして最終的に no-show timeout で処理する。逆に outer signature が valid で、その signed body が非正準、矛盾、または branch facts と不一致なら、signer に帰属する objective protocol-fraud evidence になる。

B/C が署名する CertificateBody は少なくとも

$$\begin{aligned} \text{CB} = & (\text{network}, \text{ruleset}, \text{AssignmentID}, \text{BC}, \text{H(WA)}, \\ & \text{ref}_T, T_{\text{ticket}}, \text{rosterRoot}, \text{jobLeafHash}, C_{\text{exec}}, \\ & \text{H}(P), r_{\text{exec}}, r_{\text{NISA}}, r_B, r_C, h_A). \end{aligned} \quad (41)$$

を束縛する。Full node が再導出できない field は署名対象に入れない。B/C は同じ $\text{H}(\text{CB})$ に role-separated context で署名し、target、roster、assignment、projection、execution root、evidence root の差し替えを不可能にする。System-derived PendingTicket は後の FinalCertificateID を参照するため、ticket ID をこの preimage へ循環的に含めない。

9.5 Execution-time-dependent deadline

Shape table は conformance benchmark から conservative execution bound $T_{\text{exec}}(s, \sigma)$ を ALGO-3 service block 数で固定する。Replica commitment deadline は

$$\Delta_{\text{rs}}(s, \sigma) = \lceil \alpha_{\text{exec}} T_{\text{exec}}(s, \sigma) \rceil + \Delta_{\text{net}}, \quad \alpha_{\text{exec}} \geq 3 \quad (42)$$

とする。数秒の固定 deadline は使わない。Primary opening と replica reveal の期限も shape bound と control-band propagation budget から導出する。

9.6 Blind commitment, opening, certificate

B/C は job bytes から execution を独立に開始し、primary opening 前に

$$\text{com}_R = \text{H}(\text{palw/replica-commit/v4} \parallel \text{BC} \parallel h_R \parallel P_R \parallel \rho_R) \quad (43)$$

を chain へ入れる。二つの commitment が受理された後に A が P_A, ρ_{res} と NISA witness を開く。B/C は (P_R, ρ_R) と replica-specific interval witness を開示する。

Projection mismatch は直ちに certificate を作らず、pre-PendingTicket adjudication へ入る。Objective bisection で敗者を確定し、honest primary を単一 malicious replica が破壊できないよう次順位 slot へ交代する。Certificate は

$$P_A = P_B = P_C \quad (44)$$

と全 signature、commitment opening、NISA、replica witness、target reference、roster root、bond reservation を束縛する。

No-show slash は、deadline 中に必要数の PALW service opportunity が存在し、対象 class の control band が capacity 不足で停止していない場合に限る。そうでない timeout は slash せず assignment を deferred とする。Sustained targeted censorship は hash-lane liveness failure として別途測定する。

9.7 署名後の future audit

$P_A = P_B = P_C$ と初期 witness が成立すると、B/C は CB へ署名する。この二署名が初めて同一 branch に揃った block を A_{SC} とし、SignedCertificateID を

$$\text{SCID} = \text{H}(\text{palw/signed-certificate/v1} \parallel \text{CB} \parallel \sigma_B \parallel \sigma_C) \quad (45)$$

とする。 A_{SC} 後の certificate-specific offset と未来 ALGO-3 window W_P から

$$R_{\text{post}} = \text{H}(\text{palw/post-replica-audit/v1} \parallel \text{SCID} \parallel W_P), \quad (46)$$

$$S_{\text{post}, R} = \text{Sample}(\text{H}(R_{\text{post}} \parallel h_R), \kappa_P) \quad (47)$$

を導出する。 $S_{\text{post},B}$ と $S_{\text{post},C}$ は NISA、初期 B/C witness、先行 post-audit index をすべて skip する。B/C は署名後に初めて決まる interval witness を期限内に開示しなければならない。したがって、初期 sample だけに合わせた trace または不正 ticket へ不可逆署名してから逃げる経路には、追加の objective detection route と no-show route が生じる。

Future audit は独立実行そのものを暗号学的に証明しない。Primary から正しい trace を受け取った replica と自ら replay した replica は区別できない。しかし false execution、矛盾する execution root、保持していない trace、署名後の opening refusal は検知対象となる。いずれかの replica が post-audit を拒否または失敗した場合、その replica の performance/correctness sub-lien を freeze し、次の canonical slot へ交代する。Replacement は新しい CB と署名を作り、旧 SCID や既知 sample を再利用せず、新しい SCID と未来 W_P を導出する。Escalation は E で有界であり、上限到達時は primary 無 fault の fail-closed settlement とする。

両 post-audit が通過した最初の block が

$$\text{FCID} = H(\text{palw/final-certificate/v1} \parallel \text{SCID} \parallel r_{\text{postEvidence}}) \quad (48)$$

を system-derived FinalCertificate として生成し、その後にだけ PendingTicket を生成できる。 W_P の endpoint は SCID と carrier hash の offset で分散し、Value Profile は withholding 較正を要求する。

9.8 Objective evidence と deterministic slash

Canonical replica evidence は

$$\text{OffenseID} = H(\text{palw/replica-offense/v1} \parallel \text{kind} \parallel \text{branchAnchor} \parallel \text{AssignmentID} \parallel h_R \parallel \text{evidenceDigest}) \quad (49)$$

で一意化し、

$$\text{FaultState}[\text{OffenseID}] \in \{\text{absent, frozen, settled}\} \quad (50)$$

を one-shot に遷移させる。有効 evidence が受理された同じ state transition で、該当 B_{perf} または B_{corr} を freeze し、credential を suspend し、selection capacity を 0 にし、同一 bond が支える他の pending certificate を freeze する。 Δ_{stable} 後、slash、reporter bounty、burn または Testnet-23 shadow settlement を自動実行する。Validator vote、committee approval、operator discretion は存在しない。Reorg は evidence と settlement を同じ undo record で巻き戻す。各 (AssignmentID, role, phase) は一つの terminal fault state だけを持ち、同じ fault を示す複数 evidence は alias として deduplicate する。Performance phase と correctness phase は別 lien だが、各 phase の aggregate slash は予約額を超えず、一 fault から複数 bounty を支払わない。Settlement 前に複数 kind が成立した場合は parameter package の severity priority で一意に統合する。

Timeout evidence は reporter を要せず system transition で生成する。署名済み二重 object、commitment mismatch、invalid signed witness、branch facts と矛盾する CertificateBody は誰でも ReplicaEvidence として提出できる。Bounty ownership は evidence hash と reporter key の commit-reveal で最初の valid commitment に固定し、mempool copy を防ぐ。Evidence class は control calendar と epoch reserve を持ち、bond release より前に失効しない。

Fault	Objective detection route	Enforcement	Ticket effect
No-show / reveal refusal	Assigned service turns と未消費 response ID	S_{ns} 、suspend threshold	replacement または fail closed
Authenticated malformed body	Valid outer signature と canonical decode failure	S_{proto} 、即時 suspend	object 無効、PendingTicket 無し
Equivocation	同一 assignment/role/kind への異なる二つの valid signature	S_{proto} 、capacity freeze	certificate 無効
Commitment opening mismatch	signed reveal と prior commitment の不一致	S_{proto}	replacement、再裁定
Invalid initial/post-audit witness	signed witness の Merkle/ComputeVM failure	S_{perf} 、suspend	FinalCertificate 無し
Projection mismatch 敗北	pre-PendingTicket bisection 終局	S_{mis}	honest party を維持し replacement
Invalid ticket binding	signed CB の target/roster/assignment field が branch facts と不一致	S_{proto} または B_{corr}	system ticket を生成しない
False execution certificate	permissionless dispute または future audit の終局	B/C 双方の B_{corr} 、suspend	PendingTicket reject

Invalid unauthenticated bytes、第三者が作った invalid signature、異なる branch でのみ valid な object は slash evidence ではない。Evidence は報告 branch 上で assignment と署名 obligation が存在し、offender key の valid signature または objective timeout を検証できる場合に限り受理する。

10 Layer 2: Permissionless dispute

10.1 PendingTicket

FinalCertificate の system transition は PendingTicket

$$PT = (BC, H(WA), \text{ref}_T, T_{\text{ticket}}, H(P), r_{\text{exec}}, r_{\text{certificate}}, r_{\text{evidence}}, h_A) \quad (51)$$

を生成する。PendingTicket は reward、fork-choice weight、transaction execution right を持たない。Challenge window は PendingTicket creation block から $\Delta_{\text{challenge}}$ ALGO-3 selected-chain block 開く。Lottery window 終端から数える旧 Δ_{exp} を dispute lifetime に流用しない。

Certificate pipeline の期限は

$$\Delta_{\text{cert}} = k_2 + w_2 + E\Delta_{\text{rs}} + \Delta_{\text{open}} + \Delta_{\text{reveal}} + k_3 + w_3 + \Delta_{\text{post}} + \Delta_{\text{adj}}^{\text{max}} \quad (52)$$

として独立に予算化する。 E は escalation slot 上限であり、右辺の全 phase を差し引いた後に決める。

10.2 Bounded challenge set

Challenge は commit-reveal で開始する。Commitment は ticket、interval、claim hash、reporter key、nonce を束縛し、 Δ_{cr} 内に reveal されなければ失効して bounty priority を失う。同一 (PT, interval, postClaim) は claim ID で deduplicate する。

Ticket ごとに

$$D_{\text{active}} \leq D_{\text{max}}^{\text{active}}, \quad D_{\text{total}} \leq D_{\text{max}}^{\text{total}}, \quad B_{\text{queued}} \leq B_{\text{max}}^{\text{queue}} \quad (53)$$

を強制し、epoch ごとの dispute verifier budget も resource budget に含める。Challenge commit deadline は challenge window 終端の Δ_{cr} 前とし、window 終端時に未 reveal commitment は残らない。

$D_{\text{total}} = D_{\text{max}}^{\text{total}}$ の ticket には、通常 challenge cap と C_{dispute} の外側に、固定長の ChallengeOverflowSignal を一件だけ受理する専用 service slot を設ける。Signal は ticket ID、reporter key、fresh nonce、 B_{overflow} bond を束縛し、大きな witness を持たない。期限内の signal は ChallengeOverflowReject transition で PendingTicket を fail closed に reject する。Overflow 時は B_{overflow} と work escrow E_w を burn し、既存 challenge bond は返還し、replica へ correctness slash は適用しない。Value Profile は $B_{\text{overflow}} + E_w$ が ticket 最大価値と許容 grief value の和を上回ることを要求し、別監査でこの liveness trade-off を承認しなければならない。Testnet-23 は zero-value なので safety を優先して同じ reject rule を用いる。これにより challenge-slot squatting が false maturity を生まない。

10.3 Bisection と一 step 裁定

Game は合意済み pre-state、primary post-state、challenger post-state、turn、deadline を明示する。Midpoint は $\lfloor (lo + hi)/2 \rfloor$ とし、claim pair が食い違う半区間を一意に選ぶ。Checkpoint interval まで縮めた後、Compute VM opcode index 上で同じ規則を適用し、最後に full node が一 micro-step を実行する。

Invalid move、wrong turn、malformed witness、deadline 超過は規範的な敗北 transition を持つ。Defect が一件でも成立すれば PendingTicket を reject し、work escrow と false certificate lien を決定的に settlement する。False challenge は challenger bond から defender response cost を支払う。

10.4 State-dependent maturity

PendingTicket は

$$\text{challengeClosed} \wedge D_{\text{active}} = 0 \wedge D_{\text{queued}} = 0 \wedge \text{settledStable} \quad (54)$$

のときだけ MatureTicket へ遷移する。固定高さだけで maturity を決めない。MatureTicket は

$$MT = (H(PT), r_{\text{maturityState}}, r_{\text{disputeSet}}, \text{nullifier}, r_{\text{reward}}) \quad (55)$$

である。Nullifier は

$$NF_{\text{ticket}} = H(\text{palw/ticket-nullifier/v1} \parallel H(PT)) \quad (56)$$

とし、branch-local state で一回だけ consume できる。

Testnet-23 では、式の maturity 条件を初めて満たす ALGO-3 block が deterministic system transition として zero-value record を accumulator へ追加し、nullifier を consume する。FinalizeTicket transaction、authority の online signature、使用 window は存在しないため、当選者を短い期間だけ targeted DoS して ticket を破壊する経路はない。

11 Fee, escrow, settlement

BatchCommit の attempt fee は受理時に消費し返金しない。WinAnnounce は work escrow E_w と replica-fee escrow E_{rep} を作る。 E_w は、計算せずに commitment を作り、当選後に replica capacity を購入する戦略を期待値負にする collateral である。

Value Profile では

$$E_w \geq \gamma_{\text{grief}}(C_B + C_C + C_{\text{include}} + C_{\text{verify}} + V_{\text{ticket}}), \quad \gamma_{\text{grief}} > 1 \quad (57)$$

を満たさなければならない。Testnet-23 は transferable value を持たないが、同じ式を test-credit と capacity accounting で模擬し、epoch cap により accepted fake win の被害を有界化する。

終端状態	attempt fee	replica escrow	work escrow / ticket
WinAnnounce 無し	消費	無し	無し
Invalid WinAnnounce	消費	transaction 自体を拒否	transaction 自体を拒否
Replica no-show、escalation 成功	消費	完了 replica へ支払、shadow slash を先充当	継続
Primary opening 無し / invalid	消費	commit 済み replica へ支払、残余返金	全額 burn または shadow burn
Mismatch adjudication で primary 敗北	消費	honest replica へ支払	全額 burn、PendingTicket 無し
Mismatch adjudication で replica 敗北	消費	replacement を含む honest replica へ支払	継続、replica lien slash
FinalCertificate 不成立、primary 無 fault	消費	完了分支払、残余返金	返金
Post-audit または PendingTicket challenge で execution false 確定	消費	支払済み	challenger bounty、残余 burn、ticket reject
False challenge / timeout	消費	継続	challenger bond から defender cost、残余 burn
Challenge overflow reject	消費	完了分支払	ticket reject、 $B_{\text{overflow}} + E_w$ burn、既存 challenge bond 返還
Challenge 全解決、MatureTicket	消費	B/C へ支払	Value Profile では predetermined reward、Testnet-23 は zero
MatureTicket 自動 finalization	消費	完了	system transition で nullifier consumed

すべての terminal transition は branch-local undo record を持つ。Stable depth 前の slash、refund、maturity は reorg により巻き戻る。Value Profile の subsidy、fee share、bond multiplier は consensus constant であり、変更は hard fork と独立経済監査を要する。

12 Storage, pruning, full-node cost

Winner-only data と aggregate budget により、attempt rate が job bytes retention を線形に増やすことはない。BatchCommit は固定長に近い metadata、WinAnnounce は一 job、PendingTicket は一 opening/FinalCertificate/post-audit evidence set だけを運ぶ。

Public job bytes、opening、witness、dispute data は少なくとも

$$R_{\text{arch}} \geq \Delta_{\text{challenge}} + \Delta_{\text{game}}^{\text{max}} + \Delta_{\text{stable}} \quad (58)$$

ALGO-3 selected-chain block 保持する。 $\Delta_{\text{game}}^{\text{max}}$ は service-turn 上限と class 最大 gap g_c から block 上限へ変換した値である。Non-archival node はその後 prune できるが、BatchCommit、WinAnnounce、PendingTicket、MatureTicket、dispute

result、accumulator root は保持する。古い useful-work の完全再検証には archival data が必要であり、snapshot bootstrap はこの仮定を明示する。

Full-node normal path は、weights を保持しつつ、selected-chain window、one winning hash、 $\tau + 2 + 2\kappa_r + 2\kappa_p$ interval、ML-DSA signature、bond state、dispute state、accumulator transition を検証する。Model 全体の full replay は要求しない。

13 単一 global lane と Compute Set admission

全 admitted Compute Set は一つの WinAnnounce-rate target、一つの fee rule、一つの evidence format で競争する。Model-specific target、quota、premium、validator allocation は存在しない。Work quantity は q を通じて win probability へ線形に寄与し、MatureTicket の post-discovery value は q に依存しない。

Compute Set activation package は少なくとも次を含む。

- model / tokenizer / template / weights hash、
- exact integer or fixed-point arithmetic、
- ComputeVMID、opcode cost vector、state schema、operation accumulator rule、
- homogeneous shape table、 q_{job} 、 κ_{cp} 、 T_{exec} 、padding cap、
- positive / negative conformance vectors、
- shortcut analysis。dead path と padding を含む、
- independent two-implementation test result、
- secret handling profile。 ρ_{lot} 、 ρ_{res} 、trial key は production build で Copy/Clone を禁止し、使用後 zeroize する。

Compute Set activation は少なくとも Δ_{SetLead} service block 前に manifest と artifact hash を確定し、全 reference node が complete package を取得して vector を通過した後の explicit activation fence で行う。Artifact が local に無い node は当該 Set を含む object を検証済みとして扱ってはならない。External online service は source of truth にならない。

14 Security analysis

14.1 Partial computation

$x \in [0, 1]$ を省略しようとした semantic-operation cost の割合とする。Operation accumulator により、semantically equivalent な shortcut が無い限り、省略は少なくとも一つの checkpoint interval defect を作る。Defect interval fraction を $d(x)$ とすると、相異なる $\tau_{\text{eff}} = \tau + 2\kappa_r + 2\kappa_p$ sample がすべて defect を外す確率は

$$\Pr[\text{miss}] \leq (1 - d(x))^{\tau_{\text{eff}}} \quad (59)$$

である。Shortcut が存在する場合は execution falsification ではなく work pricing failure であり、Compute Set admission と padding cap の対象となる。

正直 execution cost を c 、win path value を V 、attempt fee を f 、detection 時損失を S とすれば、部分計算の期待値は概ね

$$\text{EV}(x) \leq pV(1 - d(x))^{\tau_{\text{eff}}} - (1 - x)c - f - S(1 - (1 - d(x))^{\tau_{\text{eff}}}) \quad (60)$$

である。Value Profile は実測 $d(x)$ と collateral を用い、全 $x > 0$ で honest execution より期待値が低いことを activation gate とする。Testnet-23 は value を zero に固定するため、この経済仮定を chain ordering へ伝播させない。

14.2 Batch fake leaf と one-shot probability

Batch 中の未計算 leaf は、その leaf に割り当てられた win を開示できない。Computed leaf の trial set は fake leaf の有無で変わらず、一 BatchCommit は一 ticket だけを生成する。したがって fake fraction ϕ は成功可能 win の割合を $(1 - \phi)$ へ下げるが、正当な work trial を増やさない。

Announce 可能な複数 winning index が存在しても、 R_{rep} と R_{audit} は index と leaf から独立である。A が選べるのは、すでに正当に獲得した一 ticket に対応する job だけであり、監査・roster draw を再抽選できない。

14.3 Replica capture と utilization bound

Attacker の total bonded slot share を β 、honest slot utilization が最大 U_{max} 、attacker が全 slot を free に保つ最悪状態を考える。Free-slot share は

$$\beta_{\text{free}} \leq \frac{\beta}{\beta + (1 - \beta)(1 - U_{\text{max}})} \quad (61)$$

で上から抑えられる。これは各 credential の utilization cap を consensus が強制するためである。B/C 両方の capture は without-replacement の β_{free}^2 近傍である。さらに false ticket は NISA と replica witness を通り、challenge window を生き残る必要がある。Security report は平均 bond share でなく、実測 free-slot concentration と bribable share を用いなければならない。

14.4 Replica misconduct の検知下限と slash adequacy

Execution defect が random interval の割合 d に現れる場合、mandatory NISA、初期 replica witness、署名後 future audit による検知下限は

$$p_{\text{det}}(d) \geq 1 - (1 - d)^{\tau + 2\kappa_r + 2\kappa_p} \quad (62)$$

である。これに、permissionless dispute、objective timeout、署名済み protocol-fraud evidence が追加される。Structural ticket fraud、equivocation、validly signed malformed body は full-node rederivation により検知確率 1 である。False execution の未 sampled 部分については確率 1 を主張しない。

Value Profile は d の adversarial lower bound、watcher coverage、最大 bribe を測定し、前節の replica rationality inequality を全 supported fault class で満たさなければならない。Slash amount は gross bond でなく、reporter bounty や coalition recovery を差し引いた irrecoverable amount で評価する。Accepted evidence が slash されない discretionary path は存在しない。

14.5 Withholding

Batch endpoint と post-attestation window endpoint は、それぞれ carrier hash と BatchID または SCID で spread へ分散される。ALGO-3 miner の withholding bias は完全には消えないが、一 boundary block が同時に再抽選できる batch 数は集約 budget と spread により上限化される。Testnet-23 は window simulator で

$$\max \mathbb{E}[\text{bias benefit}] < \mathbb{E}[\text{withholding cost}] \quad (63)$$

を実測する。PALW が ordering weight を持たないため、bias benefit は zero-value ticket telemetry に限定される。

14.6 Control-loop stability

Target controller は accepted WinAnnounce を観測し、certificate と challenge delay を feedback loop から外す。 $H_{\text{half}} \geq 10\Delta_{\text{announce}}$ 、capacity clamp、epoch win cap、target clamp を同時に強制する。Randomized supply shock、replica outage、capacity saturation を含む simulation で持続振動が無いことを activation gate とする。

15 Testnet-23 初期 parameter

次は zero-value Safety Profile の開始値であり、Value Profile の値ではない。表で「ALGO-3 block」と示す horizon は selected-chain height、「service opportunity」と示す response deadline は class calendar の slot count で数える。

Parameter	初期値	目的
Hash / target	SHA3-512 / U256	output 先頭 256 bit、big-endian
k, w	64, 256	lottery future window
Δ_{anchor}	4096 ALGO-3 block	execution freshness の上限
w_{spread}	256	shared endpoint bias の分散
k_2, w_2	64, 256	announce 後 verification window
k_3, w_3	64, 64	B/C 署名後の future audit window
N_{max}	1024	homogeneous batch leaf 上限
$q_{\text{max}}^{\text{batch}}$	2^{24}	fee · loop · index 上限
$\varepsilon_{\text{multi}}$	2^{-10}	one-ticket cap の非線形性上限
Δ_{announce}	1024 ALGO-3 block	lottery 終端後の announce 期限 (約 128 秒)
C_{batch}	4096 / epoch	BatchCommit state growth 上限
C_{win}	4 / epoch	replica pipeline admission 上限
C_{dispute}	16 / epoch	new dispute admission 上限
$B_{\text{job}}^{\text{max}}$	512 KiB	WinAnnounce の selected job bytes
$B_{\text{pending}}^{\text{max}}$	8 MiB	opening、initial/post-audit witness、FinalCertificate
B_{ctrl}	8 MiB / ALGO-3 block	scheduled PALW class の最大 payload
g_{cycle}	16 ALGO-3 block	class-specific service calendar
E_B / B_{epoch}	1024 block / 64 MiB	aggregate adjunct cap (平均 64 KiB/block)
$B_{\text{epoch},c}$	3/36/16/9 MiB	job / ticket evidence / dispute / metadata reserve
τ, κ_r, κ_p	16, 4, 4	NISA、初期 replica、署名後 audit の相異なる sample
U_{max}	1/4	roster utilization hard cap
U_{soft}	1/8	capacity clamp が harden を開始する利用率
$D_{\text{max}}^{\text{active}}$	8	active dispute / ticket
$D_{\text{max}}^{\text{total}}$	32	total accepted dispute / ticket
$B_{\text{max}}^{\text{queue}}$	2 MiB / ticket	queued claim metadata 上限
Δ_{cr}	128 service opportunity	challenge commit-reveal 期限
B_{overflow}	$64E_w$	dedicated overflow signal bond
$\Delta_{\text{challenge}}$	691,200 block	8 BPS で約 24 時間 (参考値)
I_w	256 ALGO-3 block	target WinAnnounce interval (約 32 秒)
Δ_{age}	65,536 ALGO-3 block	replica lease 最小年齢 (約 2.3 時間)
$N_{\text{ns}}^{\text{max}}$	4 / 65,536 block	automatic lease suspension threshold
Δ_{setLead}	691,200 ALGO-3 block	Compute Set activation lead (約 24 時間)
Δ_{stable}	4096 ALGO-3 block	shadow settlement 安定深度 (約 8.5 分)
H_{half}	$\geq 10\Delta_{\text{announce}}$	WinAnnounce-rate controller の安定条件
S_{max}	1	Testnet-23 active Compute Set 数
$N_{\text{slots}}^{\text{max}}$	4096	replica snapshot / selection memory 上限
$B_{\text{weights}}^{\text{max}}$	genesis manifest exact size	local weight storage cap
$p_{\text{det}}^{\text{min}}$ / slash coefficients	signed parameter package	Value Profile の rationality gate。未設定なら disabled
PALW ordering weight	0	ALGO-3 fork choice を不変に保つ
PALW transferable reward	0	zero-value conformance
Real slash	disabled	shadow slash ledger のみ

T_{exec} 、 Δ_{rs} 、 Δ_{post} 、 B_{perf} 、 B_{corr} 、 S_{ns} 、 S_{proto} 、 S_{mis} 、 V_{epoch} 、fee coefficients は Compute Set benchmark と test-token policy から導出し、signed parameter package に具体値が無い限り該当 object class を disabled とする。固定の数秒値や placeholder を用いない。

16 Full-node validation order

Full node は高価な検査の前に length、version、network、ruleset、hash、budget を検査する。同一 block 内の全 PALW control transaction は canonical subnetwork order と transaction index 順に逐次適用し、budget、sequence、capacity、first-wins の結果を一意にする。規範順序は次である。

1. Wire version、network、ruleset、ComputeVMID、canonical length。
2. Parent branch の budget epoch と PALW control-band state。

3. BatchCommit signature、authority sequence、anchor、fee、 q 、one-shot state。
4. Batch 固有 lottery window、target reference、 T_{ticket} 。
5. WinAnnounce first-wins、 C_{lot} opening、winning trial、job leaf、job bytes、escrow。
6. Fixed roster root、verification window、 R_{rep} 、 R_{audit} 。
7. Capacity token、canonical assignment、performance bond reservation。
8. Replica commitment、primary opening、 C_{exec} opening、NISA、initial replica witness、projection equality。
9. CertificateBody signature、SignedCertificate、future post-audit window、post-audit witness、FinalCertificate。
10. Correctness lien reservation、objective evidence、one-shot freeze/suspend/slash state。
11. PendingTicket、challenge commit/reveal、bisection transition、queue cap、verifier budget。
12. State-dependent maturity、MatureTicket 自動 finalization、nullifier。
13. Fee、shadow slash、refund、retention、accumulator transition。

Dependency が未取得の場合、node は object を valid と推測せず pending に置く。Invalid と missing を区別し、外部 online service を参照して validity を補わない。

17 Launch gates

Public Testnet-23 は次を満たすまで公開しない。

- G1. Normative artifact.** ComputeVM package、wire contract、positive/negative vector、ruleset hash が公開され、runtime と node の独立実装が一致する。Production binary には always-win target、mock certificate、legacy wire fallback を含めない。
- G2. Determinism.** CPU/GPU vendor、long context、MoE boundary、restart、checkpoint restore、overflow、malformed tokenizer input で benign mismatch が 0。
- G3. One-shot と reorg.** Sequence、first-wins、capacity、bond、dispute、maturity、nullifier が randomized reorg、restart、clean IBD で一致する。
- G4. Attempt-side capacity.** BatchCommit rate、fee、live state、control-band occupancy が epoch cap 以内に、IBD catch-up rate が chain generation rate を上回る。
- G5. Winner-side capacity.** p99.9 job bytes、PendingTicket adjunct、verifier cost、burst 伝播、DB growth が budget 以内。
- G6. Roster.** 独立 credential、free-slot concentration、utilization $\leq U_{\text{max}}$ 、no-show、escalation、lien freeze を 30 日測定する。
- G7. Dispute.** Honest/false claim、timeout、malformed move、max active/total、queue cap を含む game を一 step まで完走する。
- G8. Control stability.** Supply shock、replica outage、capacity saturation 下で target が持続振動せず accepted WinAnnounce rate へ収束する。
- G9. Partition.** 50/50 partition、再接続、window 中 reorg、late join 後に手動裁定なしで自然収束する。
- G10. Runtime tax.** Canonical runtime の throughput、checkpoint overhead、padding ratio を公開し、admission limit 内である。
- G11. Watcher exercise.** Independent replay、challenge commit-reveal、overflow reject、bounty ownership を実 network で実証する。
- G12. Replica fault injection.** No-show、equivocation、authenticated malformed body、opening mismatch、invalid initial/post-audit witness、wrong target/roster/assignment、false certificate の全行について、検知、one-shot freeze、suspend、shadow slash、bounty、reorg rollback を実証し、invalid unauthenticated evidence が slash を起こさないことを確認する。
- G13. Bond rationality.** Value Profile は $p_{\text{det}}^{\text{min}}$ 、最大 bribe、irrecoverable slash、suspension cost を公開し、replica rationality inequality を満たす。Testnet-23 はこの gate を測定のみとし value activation は行わない。
- G14. Fail-closed profile.** fork-choice weight、transferable reward、real slash が全 binary で zero/disabled で一致する。
Value Profile と Ordering Profile は上記 gate の通過だけでは有効化されない。別仕様、外部監査、monetary policy、GHOSTDAG safety analysis、hard fork が必要である。

18 制約

1. NISA は statistical guarantee であり succinct proof ではない。
2. Compute Set の work pricing は shortcut 不在の公開仮定に依存する。Protocol は semantic usefulness や truthfulness を証明しない。
3. Full node は active weights を保持する。Active Set 数と合計 bytes は consensus cap を持つ。
4. Archival retention より古い execution の完全再検証は archival provider を要する。
5. Hash-lane targeted censorship と majority withholding は完全には除去されない。ただし Testnet-23 では PALW が ordering weight を持たない。
6. Replica certificate は独立 execution の暗号学的証明ではなく bonded attestation と sampled witness の組である。B/C が primary から正しい trace を受け取った事実自体は客観検知できず、safety は false ticket と protocol fault の検知・slash に依存する。
7. Testnet-23 の shadow bond と zero-value ticket は mainnet 経済安全性を実証しない。大額 bond による rational honesty は Value Profile の明示的仮定と activation inequality であり、Testnet-23 の結論ではない。
8. Bounded resource caps は ALGO-3 safety を守るが、資本を投じた spam が PALW ticket throughput を低下させる可能性は残る。これは ordering failure へ転化しない。
9. Compute Set の追加は coordinated activation を要し、model の陳腐化速度より upgrade が遅い戦略リスクを持つ。

19 結論

PALW v0.6.1 は、正準 LLM computation を compact な hash trial へ変換し、当選 execution だけを三層検証へ送る。Homogeneous BatchCommit と winner-only data availability は attempt-side bandwidth と資本 lock を抑え、lottery/result salt の用途分離は即時 win 検証と blind replay を両立させる。Operation accumulator は checkpoint 境界に現れない計算を execution root へ反映し、full node が weights を保持することで interval witness を実装可能な大きさへ保つ。

B/C は大額 bond を持つ経済合理的 replica として扱うが、善意には依存しない。CertificateBody の全 field binding、署名後 future audit、objective timeout、permissionless ReplicaEvidence、one-shot fault state により、不正 ticket、二重署名、opening refusal、invalid witness、false certificate を複数経路で検知する。Accepted evidence は必ず collateral freeze、credential suspension、slash settlement へ進む。Certificate は block や reward を直接作らない。PendingTicket は bounded dispute を担い、全 game が解決してから MatureTicket になる。Testnet-23 Safety Profile では MatureTicket は zero-value record であり、ALGO-3 だけが ordering と liveness を担う。これにより、未較正の useful-work economics、replica capture、witness bug、difficulty instability が fork choice や実資産へ伝播する経路を fail closed にする。

PALW が非ゼロ ordering power を得る将来 profile は、本稿の延長として黙って有効化してはならない。それは別 protocol revision であり、GHOSTDAG safety、external work recovery、monetary policy、implementation evidence を改めて監査する必要がある。

付録 A 第三次監査所見への対応表

ID	旧所見	v0.6.1 の規範修正
C-01	block-before-dispute と pre-block flow の二重規定	PendingTicket → dispute → MatureTicket の単一 flow。 Testnet-23 は block ordering を生成しない。
C-02	Δ_{exp} と challenge lifetime の不整合	Δ_{cert} 、 $\Delta_{\text{challenge}}$ 、maturity を分離。
C-03	lottery salt 公開が projection commitment を検査可能化	C_{lot} と C_{exec} 、 ρ_{lot} と ρ_{res} を用途分離。
C-04	全 attempt の job bytes と escrow が支配項	Homogeneous batching、winner-only job bytes、 q 比例 fee、escrow は WinAnnounce 時。
C-05	long-lived reserve に必要な roster が過大	B_{perf} は FinalCertificate まで、 B_{corr} は maturity まで。利用率 hard cap。

ID	旧所見	v0.6.1 の規範修正
C-06	weight tile witness が checkpoint を過細化	Full node が active weights を保持。Witness は state/path/op-code slice。
H-01	equivocation proof が執行不能	Testnet-23 は block authority と ordering ticket を持たず、該当機構を規範範囲から削除。
H-02	共通 endpoint で全 RC が同時再抽選	carrier hash と BatchID による w_{spread} offset。
H-03	announce timing で roster snapshot 選択	roster root は lottery window 終端に固定。
H-04	capacity-skip で attacker free share が増幅	global utilization hard cap と free-share bound、canonical assignment。
H-05	fixed 32 block replay deadline	$T_{\text{exec}}(s, \sigma)$ に比例する deadline。
H-06	congestion で honest slash	専用 control band、service clock pause、targeted censorship を別故障として扱う。
H-07	dead computation が checkpoint に現れない	全 semantic operation の rolling accumulator、padding cap、shortcut analysis。
H-08	dispute delay で ASERT 振動	WinAnnounce-rate controller、capacity clamp、 $H_{\text{half}} \geq 10\Delta_{\text{announce}}$ 。
H-09	短い ticket 使用窓への targeted DoS	Deterministic automatic finalization。Authority signature と use window を削除。
H-10	$\chi = 1$ が ordering weight を過小価格化	Testnet-23 の PALW ordering weight を 0 に固定。非ゼロ profile は別仕様。

付録 B B/C 検知・slash 強化の追加規範

本改訂は、B/C が大額 bond により経済合理的に不正を避けるという前提を、検証可能な protocol condition へ変換した。追加点は、(i) replica rationality inequality、(ii) valid outer signature と objective timeout による evidence completeness、(iii) target・roster・assignment・execution evidence を束縛する CertificateBody、(iv) B/C 署名後の future audit、(v) OffenseID と one-shot FaultState、(vi) fault matrix 全行の deterministic freeze・suspend・slash、(vii) invalid unauthenticated signature では slash しない attribution rule である。これらは「不正を一切見逃さない」という主張ではない。False execution の検知は sampled audit と watcher に依存し得るが、検知済み evidence を裁量で無視して slash しない経路は規範上存在しない。

付録 C Normative artifact checklist

Consensus release は次の hash を一つの RulesetID へ束縛する。

$$\text{RulesetID} = H(r_{\text{wire}} \parallel r_{\text{vm}} \parallel r_{\text{set}} \parallel r_{\text{params}} \parallel r_{\text{vectors}}) \quad (64)$$

Runtime と node は同じ input から BatchID、window、target、trial、sample、assignment、certificate、dispute、maturity、settlement を独立に再導出しなければならない。Golden vector の再生成は ruleset change であり、coordinated release と新 genesis または explicit activation fence を要する。

参考文献

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [2] A. Back, “Hashcash – A Denial of Service Counter-Measure,” 2002.
- [3] S. Haber and W. S. Stornetta, “How to Time-Stamp a Digital Document,” *Journal of Cryptology*, 1991.
- [4] R. C. Merkle, “Protocols for Public Key Cryptosystems,” IEEE Symposium on Security and Privacy, 1980.
- [5] I. Bentov, A. Gabizon, and D. Zuckerman, “Bitcoin Beacon,” 2016.
- [6] MISAKA Research, “PALW Runtime–Node Wire Contract,” protocol package, 2026.
- [7] MISAKA Research, “PALW Compute VM v2 Executable Specification,” protocol package, 2026.
- [8] MISAKA Research, “PALW Protocol Audit, Third Review,” internal report, 2026.